Module Guide for Flexin'

Team 3, Flexin'
Arjun Karthik
Shao (Daniel) Hu
Noah Goldschmied
Peter DeForest
Daniel Franze-Da Silva

November 12, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
DAG	Directed Acyclic Graph
M	Module
MG	Module Guide
OS	Operating System
R	Requirement
SC	Scientific Computing
SRS	Software Requirements Specification
Flexin'	Explanation of program name
UC	Unlikely Change
[etc. —SS]	[... —SS]

Contents

1 Revision History **i**

2 Reference Material **ii**

2.1 Abbreviations and Acronyms . . . ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**

4.1 Anticipated Changes . . . 2

4.2 Unlikely Changes . . . 2

5 Module Hierarchy **2**

6 Module Decomposition **4**

6.1 Hardware-Hiding Modules . . . 4

6.1.1 Fitness Integration Module (M1) . . . 5

6.2 Behaviour-Hiding Module . . . 5

6.2.1 Persistence Module (M2) . . . 5

6.2.2 User and Onboarding Module (M3) . . . 6

6.2.3 Activity Module (M4) . . . 6

6.2.4 Goals Module (M5) . . . 6

6.2.5 Currency Ledger Module (M6) . . . 7

6.2.6 Asset Module (M7) . . . 7

6.2.7 Transaction Order Module (M8) . . . 7

6.2.8 Portfolio Module (M9) . . . 8

6.2.9 Leaderboard and Friends Module (M10) . . . 8

6.2.10 Progress Reporting Module (M11) . . . 8

6.2.11 UI Screens Module (M12) . . . 8

6.2.12 Notification Module (M13) . . . 9

6.2.13 Market Simulation Module (M14) . . . 9

6.2.14 API Client Module (M15) . . . 9

6.3 Software Decision Module . . . 10

6.3.1 Authentication and Security Module (M16) . . . 10

7 Connection Between Requirements and Design **10**

7.1 Traceability Matrix . . . 10

8 Use Hierarchy Between Modules **11**

9 User Interfaces **12**

10 Design of Communication Protocols **12**

11 Database Design **12**

12 Timeline **12**

13 Generative AI Use Disclosure **13**
13.1 AI Tools Used 13
13.2 How and Where Used 13
13.3 Verification of AI-Generated Content 13

List of Tables

1 Module Hierarchy 4
2 Trace Between Requirements and Modules 11
3 Trace Between Anticipated Changes and Modules 11

List of Figures

1 Use hierarchy among modules 12

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (**?**). We advocate a decomposition based on the principle of information hiding (**?**). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is valuable in SC, where modifications are frequent, especially during initial development as the solution space is explored.

Our design follows the rules layed out by **?**, as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (**?**). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 gives a detailed description of the modules. Section 7 specifies the connections between the software requirements and the modules, and Section 7.1 within it includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 8 describes the use relation between modules.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

AC1: The specific hardware on which the software is running.

AC2: The format of the initial input data.

...

[Anticipated changes relate to changes that would be made in requirements, design or implementation choices. They are not related to changes that are made at run-time, like the values of parameters. —SS]

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

UC1: Input/Output devices (Input: File and/or Keyboard, Output: File, Memory, and/or Screen).

...

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

The mobile application and the backend service are treated as a single module hierarchy rather than two separate subsystems. The boundary between them is the API Client Module (M15): mobile modules reach backend behaviour only through it, so the transport between the two can change without affecting either side.

M1: Fitness Integration Module

M2: Persistence Module

M3: User and Onboarding Module

M4: Activity Module

M5: Goals Module

M6: Currency Ledger Module

M7: Asset Module

M8: Transaction Order Module

M9: Portfolio Module

M10: Leaderboard and Friends Module

M11: Progress Reporting Module

M12: UI Screens Module

M13: Notification Module

M14: Market Simulation Module

M15: API Client Module

M16: Authentication and Security Module

Level 1	Level 2
Hardware-Hiding Module	Fitness Integration Module
Behaviour-Hiding Module	Persistence Module
	User and Onboarding Module
	Activity Module
	Goals Module
	Currency Ledger Module
	Asset Module
	Transaction Order Module
	Portfolio Module
	Leaderboard and Friends Module
	Progress Reporting Module
	UI Screens Module
	Notification Module
	Market Simulation Module
	API Client Module
Software Decision Module	Authentication and Security Module

Table 1: Module Hierarchy

6 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by **?**. The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *Flexin'* means the module will be implemented by the Flexin' software.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

6.1 Hardware-Hiding Modules

Secrets: How the system interacts with physical devices: the phone's sensors and the wearables that record the user's activity, and the platform services that expose their readings.

Services: Serves as a virtual layer over the devices that physically measure the user's activity, so the rest of the system can read activity data without knowing which device recorded it or how that device is accessed. Only modules that directly manipulate or read from hardware belong here; modules that merely use storage or the network, where the system does not choose or control the physical device, are behaviour-hiding.

Implemented By: –

6.1.1 Fitness Integration Module (M1)

Secrets: How each fitness data provider (Health Connect, Strava, Garmin) is accessed: its SDK or web API, its permission model, its record formats, how manually entered records are recognised and excluded, and how step counts recorded by several sources are combined without double counting.

Services: Reports whether a provider is available on the device, requests read permission for the data the system needs, and returns workouts and daily step totals since a given date in a common, provider-independent format. Each record carries a stable key so that the same activity is never reported as new twice.

Implemented By: Flexin' (mobile), using the provider SDKs and APIs

Type of Module: Abstract Data Type (one instance per provider, all sharing a single interface)

6.2 Behaviour-Hiding Module

Secrets: The contents of the required behaviours, and how the data and services those behaviours depend on are represented within Flexin'.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents, together with the modules that are specific to Flexin' and support that behaviour: storing the system's entities, simulating the market and communicating between the mobile app and the backend. Storage is behaviour-hiding rather than hardware-hiding because the system does not choose or manipulate where its data is physically stored; it only decides what is stored and how it is organised. The programs in this module will need to change if there are changes in the SRS.

Implemented By: –

6.2.1 Persistence Module (M2)

Secrets: The database management system, the database schema, and how entities are mapped to tables, queried and migrated between versions.

Services: Stores and retrieves the system's persistent entities (users, activities, goals, ledger entries, assets, prices, holdings, transactions, leaderboard groups and notification preferences) on behalf of the backend modules.

Implemented By: Flexin' (backend), using Spring Data JPA, Liquibase and MySQL

Type of Module: Abstract Object

6.2.2 User and Onboarding Module (M3)

Secrets: How user accounts and onboarding questionnaire responses are represented and validated.

Services: Registers new users with a unique email address, looks up and updates user profiles, and records the user's onboarding answers about their fitness level and financial background.

Implemented By: Flexin' (backend)

Type of Module: Abstract Object

6.2.3 Activity Module (M4)

Secrets: How synced activity is represented, stored and deduplicated, and which records are eligible for a reward.

Services: Accepts workouts and daily step totals synced from the mobile app, ignores records that have already been received, reports a user's recorded activity over a given period, and tells the Currency Ledger Module when new eligible activity should be rewarded.

Implemented By: Flexin' (backend)

Type of Module: Abstract Object

6.2.4 Goals Module (M5)

Secrets: How fitness goals are represented and how progress towards a goal is measured from recorded activity.

Services: Lets users create, update and cancel fitness goals, measures progress towards each active goal from the activity reported by the Activity Module, marks a goal complete when its target is reached, and tells the Currency Ledger Module when a completed goal should be rewarded.

Implemented By: Flexin' (backend)

Type of Module: Abstract Object

6.2.5 Currency Ledger Module (M6)

Secrets: The reward rule that converts activity into virtual currency (rates, bonuses and caps), and how the ledger of credits and debits is represented.

Services: Credits currency for newly rewarded activity and goals, debits currency for purchases, credits the proceeds of sales, and reports each user's current balance and history of entries. A balance can never become negative, and currency can only be created by fitness activity or investment returns.

Implemented By: Flexin' (backend)

Type of Module: Abstract Object

6.2.6 Asset Module (M7)

Secrets: How the simulated assets and asset types (stocks, bonds, ETFs and funds) are defined and represented, and how many units of each asset are available in the simulated market.

Services: Lists the assets that can be traded, describes each asset (symbol, name and asset type), and reports and updates the quantity of each asset available. A new asset type can be added through new asset definitions alone, without changes to the trading logic.

Implemented By: Flexin' (backend)

Type of Module: Abstract Object

6.2.7 Transaction Order Module (M8)

Secrets: How buy and sell orders are validated and executed, and how the record of executed transactions is represented.

Services: Executes buy and sell orders at the asset's current simulated price, rejects orders the user cannot afford or holdings they do not own, ensures a repeated order request is executed only once, debits or credits the user's balance through the Currency Ledger Module, updates the user's holdings through the Portfolio Module, and records a transaction for every executed trade. An order either completes in full or has no effect.

Implemented By: Flexin' (backend)

Type of Module: Abstract Object

6.2.8 Portfolio Module (M9)

Secrets: How a user's holdings and their cost are represented, and how portfolio value and gains or losses are calculated.

Services: Maintains one portfolio per user, reports the quantity of each asset the user holds, and reports the portfolio's total value and its gains or losses at current simulated prices. Holdings change only when the Transaction Order Module executes a trade.

Implemented By: Flexin' (backend)

Type of Module: Abstract Object

6.2.9 Leaderboard and Friends Module (M10)

Secrets: How friend lists and leaderboard groups are stored, and how members of a group are ranked.

Services: Lets users manage a list of friends, create private leaderboard groups and invite friends to them, join and leave public or private groups, and view each group's members ranked by progress towards their goals. Leaderboard standings never award currency.

Implemented By: Flexin' (backend)

Type of Module: Abstract Object

6.2.10 Progress Reporting Module (M11)

Secrets: Which figures are combined into a progress summary and how they are aggregated over a reporting period.

Services: Produces summaries of a user's fitness activity, goal completion, currency earned, portfolio performance and leaderboard standings by querying the modules that own that data.

Implemented By: Flexin' (backend)

Type of Module: Library

6.2.11 UI Screens Module (M12)

Secrets: The layout and visual style of each screen (colours, typography and spacing), the navigation between screens, and how loading, empty and error states are presented to the user.

Services: Presents the sign-in and onboarding flows, the dashboard, the activity screen, the exchange and asset screens, the order screen and the profile screen, and turns user input into calls on the Fitness Integration and API Client modules.

Implemented By: Flexin' (mobile), using React Native and Expo Router

Type of Module: Library

6.2.12 Notification Module (M13)

Secrets: Which events produce a notification, how each message is worded, how the user's notification preferences, quiet hours and daily limit are enforced, and how notifications are delivered to the device.

Services: Sends workout reminders, reward and achievement notices and simulated market notices to the user. It lets users turn each kind of notification on or off and set quiet hours. It sends nothing during quiet hours or for a kind the user has turned off, and by default sends no more than MaxDailyNotifications goal-related notifications per day. If notifications are disabled or delivery fails, the failure is handled without affecting the rest of the system.

Implemented By: Flexin' (backend and mobile), using the Android notification service

Type of Module: Abstract Object

6.2.13 Market Simulation Module (M14)

Secrets: The algorithm used to simulate price movement over time and how it is calibrated for each asset type.

Services: Provides the current price of each asset defined by the Asset Module and its price history, and updates prices on a schedule.

Implemented By: Flexin' (backend)

Type of Module: Abstract Object

6.2.14 API Client Module (M15)

Secrets: The backend's address, the transport protocol and message format, how credentials are attached to requests, and how network failures are retried or reported.

Services: Gives the mobile modules one typed function per backend operation (for example syncing activity, reading the balance, listing assets and placing an order), so they never build requests or parse responses themselves.

Implemented By: Flexin' (mobile)

Type of Module: Library

6.3 Software Decision Module

Secrets: General-purpose design decisions based on mathematical theorems, established algorithms or programming considerations. The secrets of these modules are *not* described in the SRS and do not depend on anything specific to Flexin'.

Services: Includes abstract data structures and algorithms that do not provide direct interaction with the user and that could be reused, largely unchanged, in other projects.

Implemented By: –

6.3.1 Authentication and Security Module (M16)

Secrets: The authentication mechanism, how credentials are hashed and verified, how sessions or tokens are issued, and which requests require an authenticated user.

Services: Verifies a user's credentials, identifies the user making each request to the backend, and rejects requests that are unauthenticated or that try to access another user's data.

Implemented By: Flexin' (backend), using Spring Security

Type of Module: Library

7 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

[The intention of this section is to document decisions that are made "between" the requirements and the design. To satisfy some requirements, design decisions need to be made. Rather than make these decisions implicit, they are explicitly recorded here. For instance, if a program has security requirements, a specific design decision may be made to satisfy those requirements with a password. —SS]

7.1 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Req.	Modules
R1	M??, M??, M??, M??
R2	M??, M??
R3	M??
R4	M??, M??
R5	M??, M??, M??, M??, M??, M??
R6	M??, M??, M??, M??, M??, M??
R7	M??, M??, M??, M??, M??
R8	M??, M??, M??, M??, M??
R9	M??
R10	M??, M??, M??
R11	M??, M??, M??, M??

Table 2: Trace Between Requirements and Modules

AC	Modules
AC1	M??
AC2	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??

Table 3: Trace Between Anticipated Changes and Modules

8 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. **?** said of two programs A and B that A *uses* B if correct execution of B may be necessary for A to complete the task

described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of B. Figure 1 illustrates the use relation between the modules. It can be seen that the graph is a directed acyclic graph (DAG). Each level of the hierarchy offers a testable and usable subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

[The uses relation is not a data flow diagram. In the code there will often be an import statement in module A when it directly uses module B. Module B provides the services that module A needs. The code for module A needs to be able to see these services (hence the import statement). Since the uses relation is transitive, there is a use relation without an import, but the arrows in the diagram typically correspond to the presence of import statement. —SS]

[If module A uses module B, the arrow is directed from A to B. —SS]

Figure 1: Use hierarchy among modules

9 User Interfaces

[Design of user interface for software and hardware. Attach an appendix if needed. Drawings, Sketches, Figma —SS]

10 Design of Communication Protocols

[If there is communication between hardware and software, it may be necessary to explain the design of the communication protocol. This section should be filled in with NA if it is not appropriate for a given project. —SS]

11 Database Design

[If a database is a significant component of the project, the database design should be summarized here. This section should be filled in with NA if it is not appropriate for a given project. —SS]

12 Timeline

[Schedule of tasks and who is responsible —SS]

[You can point to GitHub if this information is included there —SS]

13 Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]